

Chapter 2

RISC-V Trace Encoder (TRACE)

The high-performance CPU (HP CPU) of ESP32-P4 supports the instruction trace interface through the trace encoder. The trace encoder connects to HP CPU's instruction trace interface, compresses the information into smaller packets, and then stores the packets into internal SRAM (see Chapter 7 *System and Memory*).

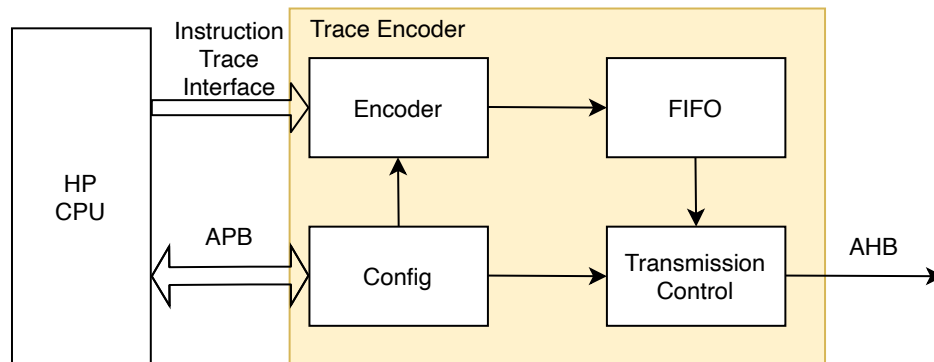


Figure 2.0-1. Trace Encoder Overview

2.1 Terminology

To better illustrate the functions of the RISC-V Trace Encoder, the following terms are used in this chapter.

hart	RISC-V hardware thread
branch	an instruction which conditionally changes the execution flow
uninferable discontinuity (updiscon)	a program counter change that can not be inferred from the program binary alone
delta	a change in the program counter that is other than the difference between two instructions placed consecutively in memory
trap	the transfer of control to a trap handler caused by either an exception or an interrupt
qualification	an instruction that meets the filtering criteria passes the qualification, and will be traced
te_inst	the name of the packet type emitted by the encoder
retire	the final stage of executing an instruction, when the machine state is updated
EPC	exception program counter

2.2 Introduction

In complex systems, understanding program execution flow is not straightforward. This may be due to a number of factors, for example, interactions with other cores, peripherals, real-time events, poor implementations, or some combination of all of the above.

It is hard to use a debugger to monitor the program execution flow of a running system in real time, as this is intrusive and might affect the running state. However, it is important to provide the visibility of program execution.

That is where instruction trace comes in, which provides trace of the program execution.

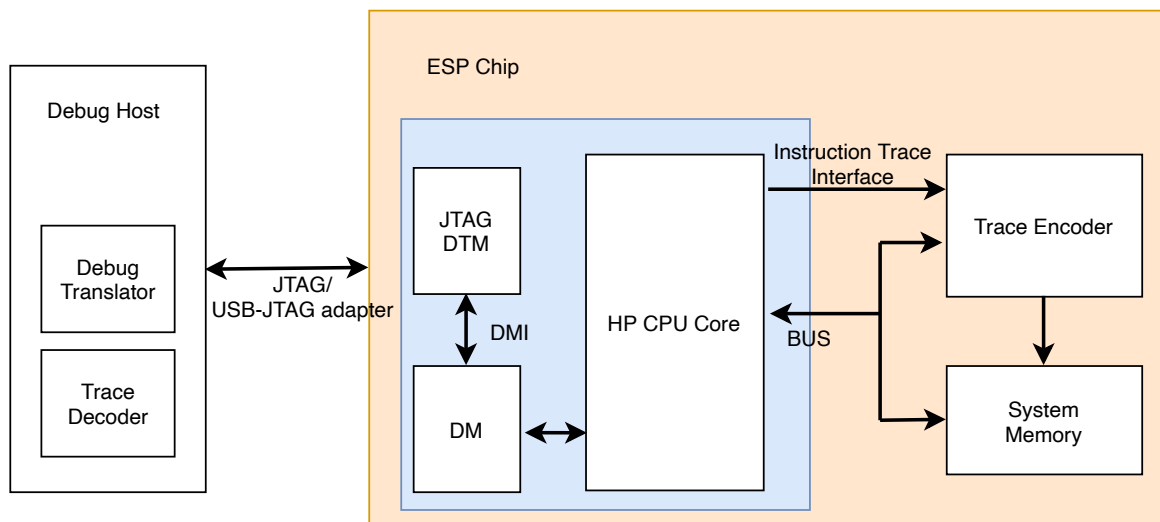


Figure 2.2-1. Trace Flow Overview

Figure 2.2-1 shows the instruction delta trace flow. Each HP Core has a private encoder that can work independently, and the trace flow of each core is the same:

- The HP CPU core provides an instruction trace interface that outputs the instruction information executed by the HP CPU. Such information includes instruction address, instruction type, etc. For more details about ESP32-P4 HP CPU's instruction trace interface, please refer to Chapter 1 [High-Performance CPU](#).
- The trace encoder collects the relevant instruction trace information from HP CPU's instruction trace interface, compresses the information into lower bandwidth packets, and then stores the packets in system memory.
- The debugger (debug host) can dump the trace packets from the system memory via JTAG or USB Serial/JTAG, and use a decoder to decompress and reconstruct the program execution flow. The trace decoder, usually software on an external PC, takes in the trace packets and reconstructs the program instruction flow with the program binary that runs on the originating hart. This decoding step can be done offline or in real time while the hart is executing.

2.3 Features

- Compatible with Efficient Trace for RISC-V Version 2.0. See Table 2.3-1 for the implemented parameters
- Support for delta address mode and full address mode
- Support for a filter unit
- Support for notifying an instruction address via debug trigger or filter unit
- Support for the following sideband signals to control trace data flow:
 - Support for debugging trigger to start or end encoder
 - When the hart is halted, the encoder can report the last packet and then stop
 - When the hart is reset, the encoder can report the last packet and then stop
 - Support for stalling the hart when FIFO is almost full
- Arbitrary address range of the trace memory size
- Configurable synchronization modes:
 - Synchronization counter counts by packet
 - Synchronization counter counts by cycle
 - Synchronization counter can be disabled
- Trace lost status to indicate packet loss
- Automatic restart after packet loss
- Memory writing in the loop or non-loop mode
- Support two interrupts:
 - Triggered when the packet size exceeds the configured memory space
 - Triggered when a packet is lost
- FIFO (128 × 8 bits) to buffer packets
- AHB burst transmission with configurable burst length

Table 2.3-1. Trace Encoder Parameters

Parameter Name	Value	Description
arch_p	0	Initial version
bpred_size_p	0	Branch prediction mode is not supported
cache_size_p	0	Jump target cache mode is not supported
call_counter_size_p	0	Implicit return mode is not supported
ctype_width_p	0	Width of the ctype bus
context_width_p	0	Width of context bus
ecause_width_p	6	Width of exception cause
ecause_choice_p	0	Multiple choice is not supported
f0s_width_p	0	Format 0 packets are not supported
filter_context_p	0	Filtering on context is not supported

Parameter Name	Value	Description
filter_excint_p	1	Filtering on exception cause or interrupt is supported
filter_privilege_p	1	Filtering on privilege is supported
filter_tval_p	1	Filtering on trap value is supported
iaddress_lsb_p	1	Compressed instructions are supported
iaddress_width_p	32	The instruction bus is 32-bit
iretire_width_p	1	Width of the iretire bus
ilastsize_width_p	0	Width of the ilastsize
itype_width_p	3	Width of the itype bus
nocontext_p	1	Exclude context from te_inst packets
notime_p	1	Exclude time from te_inst packets
privilege_width_p	1	Only machine and user mode are supported
retires_p	1	Maximum number of instructions that can be retired per block
return_stack_size_p	0	Implicit return mode is not supported
sijump_p	0	Sequentially inferable jump mode is not supported
taken_branches_p	1	Only one instruction retired per cycle
impdef_width_p	0	Not implemented

For detailed descriptions of the above parameters, please refer to the Efficient Trace for RISC-V Version 2.0 > Chapter Parameters and Discovery.

2.4 Architectural Overview

This chapter mainly introduces the implementation details of ESP32-P4's trace encoder.

As shown in Figure 2.0-1, the trace encoder contains an encoder, a FIFO, a register configuration module, and a transmission control module.

The encoder receives the HP CPU's instruction information via the instruction trace interface, compresses it into different packets, and writes it to the internal FIFO.

The transmission control module writes the data in the FIFO to the internal SRAM through the AHB bus.

The FIFO is 128 deep and 8-bit wide. When the memory bandwidth is insufficient, the FIFO may overflow, and packet loss occurs. If a packet is lost, the encoder will send a packet to tell that a packet is lost, stop working until the FIFO is empty, and also trigger an interrupt meanwhile if the interrupt is enabled.

2.5 Functional Description

2.5.1 Synchronization

In order to make the trace robust there must be regular synchronization points within the trace. Synchronization is accomplished by sending a full valued instruction address. When the synchronization counter value reaches the value of the [TRACE_RESYNC_PROLONGED](#) field of the [TRACE_RESYNC_PROLONGED_REG](#) register, the encoder will send a synchronization packet (format 3 subformat 0, see Section 2.6.3.1).

The synchronization counter is configured via [TRACE_RESYNC_MODE](#):

- 0: Disable the synchronization counter
- 1: Invalid. No effect
- 2: Synchronization counter counts by packet
- 3: Synchronization counter counts by cycle

You can adjust the trace bandwidth by increasing the value of [TRACE_RESYNC_PROLONGED](#) to reduce the frequency of sending synchronization packets, thereby reducing the bandwidth occupied by packets.

2.5.2 Address Mode

ESP32-P4 supports two address modes: delta address mode and full address mode.

- Delta address mode (default): In delta address mode, addresses are encoded as the difference between the actual address of the current instruction and the actual address of the instruction reported in the previous packet that contained an address. This differential encoding requires fewer bits than the full address, and thus results in more efficient trace compression.
- Full address mode: In full address mode, all addresses in the trace are encoded as absolute addresses instead of in differential form. This kind of encoding is always less efficient, but it can be a useful debugging aid for software decoder developers. This mode is enabled by setting [TRACE_FULL_ADDRESS](#).

2.5.3 Optional Sideband Signals

The Efficient Trace for RISC-V Version 2.0 > Chapter Optional Sideband Signals has provided some optional sideband signals for encoder control. ESP32-P4 has implemented the following functions:

Table 2.5-1. Optional sideband signals

Signal	Function
trigger[2:0]	• A pulse on bit 0 will cause the encoder to start tracing, and continue until further notice, subject to other filtering criteria also being met. • A pulse on bit 1 will cause the encoder to stop tracing until further notice. • A pulse on bit 2 will cause the encoder to report a specific address. This function is enabled by setting TRACE_DM_TRIGGER_ENA. The trigger signal is asserted upon a trigger match. For example, if the action field of mcontrol is 2, while the trigger matched it will assert trigger bit0; if the action is 3, while the trigger matched it will assert trigger bit1. For details, please refer to Table 2.8-1.
halted	Hart is halted. Upon assertion, the encoder will output a packet to report the address of the last instruction retired before halting, followed by a support packet to indicate that tracing has stopped. Upon deassertion, the encoder will start tracing again, commencing with a synchronization packet. This function is enabled by setting TRACE_HALT_ENA.

Signal	Function
reset	Hart is in reset. Behavior is as described above for halted.
stall	Stall request to hart. Some applications may require lossless trace, which can be achieved by using this signal to stall the hart if the trace encoder is unable to output a trace packet (internal FIFO almost full). This function is enabled by setting TRACE_STALL_ENA .

2.5.4 Filtering

Filtering provides a mechanism to control the criteria for encoder to produce a trace. For example, it may be desirable to trace:

- When the instruction address is within a particular range
- Starting from one instruction address and continue until a second instruction address
- For one or more specified privilege levels
- Exception and/or interrupt handlers for specified exception causes

The filtering unit has a filter and a comparator unit.

Each comparator unit is actually a pair of comparators (Primary and Secondary, or P, S) allowing a bounded range to be matched with a signal unit if required, and offers:

- Input selected from iaddress and tval
- A range of arithmetic options (<, >, =, !=, etc.) independently selectable for each comparator
- Secondary match value may be used as a mask for the primary comparator
- The two comparators can be combined in several ways: P , $P \& \& S$, $!(P \& \& S)$, latch (set on P clear on S)
- Each comparator can also be used to explicitly report a particular instruction address

Each filter can specify filtering against instruction from the hart, and offers:

- 1 run-time selectable comparator units to match
- Multiple choice selection for priv and cause inputs
- Select matching for interrupt

2.5.5 Anchor Tag

Since the length of data packets is variable, in order to identify boundaries between data packets when packed packets are written to memory, the ESP32-P4 encoder inserts zero bytes between data packets:

- The maximum packet length is 13 bytes, so a sequence of at least 14 zero bytes cannot occur within a normal packet. Therefore, the first non-zero byte seen after a sequence of at least 14 zero bytes must be the first byte of a packet.
- Every time when 128 packets are transmitted, the encoder writes 14 zero bytes to the memory partition boundary as anchor tags.

2.5.6 Memory Writing Mode

When writing the trace memory, the size of the trace packets might exceed the capacity of the memory. In this case, you can choose whether to wrap around the trace memory or not by configuring the memory writing mode:

- Loop mode: When the size of the trace packets exceeds the capacity of the trace memory (namely when `TRACE_MEM_CURRENT_ADDR_REG` reaches the value of `TRACE_MEM_END_ADDR_REG`), the trace memory is wrapped around, so that the encoder loops back to the memory's starting address `TRACE_MEM_START_ADDR_REG`, and old data in the memory will be overwritten by new data.
- Non-loop mode: When the size of the trace packets exceeds the capacity of the trace memory, the trace memory is not wrapped around. The encoder stops at `TRACE_MEM_END_ADDR_REG`, and old data will be retained.

2.5.7 Automatic Restart

When packets are lost due to FIFO overflow, the encoder will stop working and need to be resumed by software manually or restarted by hardware automatically. If the `TRACE_RESTART_ENA` bit of `TRACE_TRIGGER_REG` is set, once the FIFO is empty, the encoder can automatically be restarted.

If the automatic restart feature is enabled, the encoder will be restarted in any case. Therefore, to disable the encoder, the automatic restart feature must be disabled first by clearing the `TRACE_RESTART_ENA` bit of the `TRACE_TRIGGER_REG` register.

2.6 Encoder Output Packets

This section mainly introduces ESP32-P4 trace encoder output packet format.



Figure 2.6-1. Trace packet Format

A packet includes header, index and payload. Header, index and payload are transmitted sequentially in bit stream form, from the fields listed at the top of tables below to the fields listed at the bottom. If a field consists of multiple bits, then the least significant bit is transmitted first.

2.6.1 Header

Header is 1-byte long. The format of header is shown in Table 2.6-1.

Table 2.6-1. Header Format

Field	Bits	Description	Value
length	5	Length of whole packet	4 ~ 13
flow	2	Reserved	0
timestamp	1	Reserved	0

2.6.2 Index

Index has 2 bytes. The format of index is shown in Table 2.6-2.

Table 2.6-2. Index Format

Field	Bits	Description	Value
index	16	The index of each packet	0~65536

2.6.3 Payload

The length of payload ranges from 1 byte to 10 bytes.

2.6.3.1 Format 3 Packets

Format 3 packets are used for synchronization, and report supporting information. There are 4 subformats defined in the specification. ESP32-P4 only supports 3 of them.

Format 3 Subformat 0 - Synchronization

This packet contains all the information the decoder needs to fully identify an instruction. It is sent for the first traced instruction (unless that instruction also happens to be the first one in an exception handler), and when synchronization has been scheduled by expiry of the synchronization timer. The payload length is 5 bytes.

Table 2.6-3. Packet format 3 subformat 0

Field name	Bits	Description
format	2	11 (sync): Synchronization
subformat	2	00 (start): Start of tracing, or resync
branch	1	Set to 0 if the address points to a branch instruction, and the branch was taken. Set to 1 if the instruction is not a branch or if the branch is not taken.
privilege	1	The privilege level of the reported instruction
address	31	Full instruction address. The address must be left shifted 1 bit in order to recreate the original byte address.
sign_extend	3	Reserved

Format 3 Subformat 1 - Exception

This packet also contains all the information the decoder needs to fully identify an instruction. It is sent following an exception or interrupt, and includes the cause, the 'trap value' (for exceptions), and the address

of the trap handler or of the exception itself. The length is variable: if the interrupt field is 1, the tvalepc field is omitted and the length is 6 bytes, otherwise the length is 10 bytes.

Table 2.6-4. Packet format 3 subformat 1

Field name	Bits	Description
format	2	11 (sync): Synchronization
subformat	2	01 (exception): Exception cause and trap handler address
branch	1	Set to 0 if the address points to a branch instruction, and the branch was taken. Set to 1 if the instruction is not a branch or if the branch is not taken.
privilege	1	The privilege level of the reported instruction
ecause	6	Exception cause
interrupt	1	Interrupt
theadr	1	When set to 1, the address field points to the trap handler address. When set to 0, the address field points to the EPC (exception program counter) for an exception at the target for an updiscon, and is undefined for other exceptions and interrupts.
address	31	Full instruction address. The value of this field must be left shifted 1 bit in order to recreate the original byte address.
tvalepc	32	Value from appropriate utval/stval/mtval CSR (control/status register). Omitted if the interrupt is 1
sign_extend	3	Reserved

Format 3 Subformat 3 - Support

This packet provides supporting information to aid the decoder. It is issued when the trace is ended, filter match is failed, or a packet is lost. The length is 2 bytes.

Table 2.6-5. Packet format 3 subformat 3

Field name	Bits	Description
format	2	11 (sync): Synchronization
subformat	2	11 (support): Supporting information for the decoder
enable	1	Indicates if the encoder is enabled
encoder_mode	1	Always 0. Only supported by branch trace
qual_status	2	Indicates qualification status: • 00 (no_change): No change to filter qualification • 01 (ended_rep): Qualification ended, preceding instruction sent explicitly to indicate last qualification instruction • 10 (trace lost): One or more packets lost • 11 (ended_upd): Qualification ended, preceding te_inst would have been sent anyway due to an updiscon, even if wasn't the last qualified instruction

Field name	Bits	Description
ioptions	6	Indicates optional modes: - sequentially inferred jumps (Bits 5): When set to 1, the targets of sequentially inferable jumps will not be reported. (This feature is not implemented, so bit 5 should always be 0) - branch prediction (Bit 4): When set to 1, the branch prediction is enabled. (This feature is not implemented, so bit 4 should always be 0) - jump target cache (Bit 3): When set to 1, the jump target cache is enabled. (This feature is not implemented, so bit 3 should always be 0) - full address (Bit 2): 0: delta address mode 1: full address mode - implicit exception (Bit 1): Exclude address from format 3 subformat 1 packet if trap vector can be determined from ecause. (This feature is not implemented, so bit 1 should always be 0) - implicit return (Bit 0): When set to 1, function return addresses will not be reported. (This feature is not implemented, bit 0 should always be 0)
sign_extend	2	Reserved

2.6.3.2 Format 2 Packets

This packet contains only an instruction address, and is used when the address of an instruction must be reported (for example if the instruction is the target for an updiscon, or is the last instruction before exception), and there is no reported branch information.

The length is variable if delta address mode is enabled, ranging from 5 bytes to 8 bytes. The address field can be 8/16/24/32 bits, inferred from the packet length. For example, if the header length is n bytes, the address bits are $n - 4$ bytes. A sign bit is required to extend the address field to 32 bits.

Table 2.6-6. Packet format 2

Field name	Bits	Description
format	2	10 (addr-only): No branch information
notify	1	If the value of this bit is different from the MSB of address, it indicates that this packet is reporting an instruction that is not the target of an uninferable discontinuity because a notification was requested via trigger[2] or a filter matched.
updiscon	1	If the value of this bit is different from notify, it indicates that this packet is reporting the instruction following an uninferable discontinuity and is also the instruction before an exception, privilege change or resync.

Field name	Bits	Description
zero_extend	4	Reserved
address	Variable	Delta instruction address. These bits can be 8/16/24/32.

2.6.3.3 Format 1 Packets

This packet includes branch information, and is used when either the branch information must be reported (for example because the branch map is full), or when the address of instruction must be reported, and there has been at least one branch since the previous packet.

The packet length is variable if delta address mode is enabled, ranging from 5 bytes to 12 bytes. The address field can be 8/16/24/32 bits, inferred from the packet length. For example:

- The header length is n bytes
- The value of the format field is 1
- The value of the branch field is:
 - 0: the packet is format 1 without address
 - 1: branch_map is 1 bit wide, zero_ext is 6 bits wide, and address is $n - 5$ bits wide
 - 2 ~ 3: branch_map is 3 bits wide, zero_ext is 4 bits wide, and address is $n - 5$ bits wide
 - 4 ~ 7: branch_map is 7 bits wide, zero_ext is 0 bits wide, and address is $n - 5$ bits wide
 - 8 ~ 15: branch_map is 15 bits wide, zero_ext is 0 bits wide, and address is $n - 6$ bits wide
 - 16 ~ 31: branch_map is 31 bits wide, zero_ext is 0 bits wide, and address is $n - 8$ bits wide

Format 1 - address, branch_map

The length is variable.

Table 2.6-7. Packet format 1 with address

Field name	Bits	Description
format	2	01: Includes branch information
branches	5	Number of valid bits branch_map. The number of bits of branch_map is determined as follows: • 0: (cannot occur for this format) • 1: 1 bit • 2 ~ 3: 3 bits • 4 ~ 7: 7 bits • 8 ~ 15: 15 bits • 16 ~ 31: 31 bits For example if branches = 12, branch_map is 15-bit long, and the 12 LSBs are valid.

Field name	Bits	Description
branch_map	Variable	An array of bits indicating whether branches are taken or not. Bit 0 represents the oldest branch instruction executed. For each bit: • 0: branch taken • 1: branch not taken The field Bits is variable and determined by the branches field.
notify	1	If the value of this bit is different from the MSB of address, it indicates that this packet is reporting an instruction that is not the target of an uninferable discontinuity because a notification was requested via trigger[2] or a filter matched.
updiscon	1	If the value of this bit is different from notify, it indicates that this packet is reporting the instruction following an uninferable discontinuity and is also the instruction before an exception, privilege change or resync.
zero_ext	Variable	The length of this field is determined by branches as follows: • 1: 6 bits • 2 ~ 3: 4 bits • 4 ~ 31: 0 bits
address	Variable	Delta instruction address. These bits can be 8/16/24/32

Format 1 - no address, branch_map

The length is 5 bytes.

Table 2.6-8. Packet format 1 without address

Field name	Bits	Description
format	2	01: includes branch information
branches	5	Number of valid bits in branch_map. The length of branch_map is 31 bits. Only 0 valid.
branch_map	31	An array of bits indicating whether branches are taken or not. Bit 0 represents the oldest branch instruction executed. For each bit: • 0: branch taken • 1: branch not taken
sign_extend	2	Reserved

2.7 Interrupt

ESP32-P4's CORE_n_TRACE (i.e., the trace encoder of core 0 or core 1) can generate the CORE_n_TRACE_INTR interrupt signal that will be sent to the [Interrupt Matrix](#).

There are several internal interrupt sources from CORE_n_TRACE that can generate the above interrupt signal. The interrupt sources from CORE_n_TRACE are listed with their trigger conditions and the resulted interrupt signals in Table 2.7-1.

Table 2.7-1. CORE_n_TRACE's Internal Interrupt Sources

Internal Interrupt Source	Trigger Condition	Interrupt Signal
TRACE_MEM_FULL_INTR	Triggered when the packet size exceeds the capacity of the trace memory, namely when TRACE_MEM_CURRENT_ADDR_REG reaches the value of TRACE_MEM_END_ADDR_REG . If necessary, this interrupt can be enabled to notify the HP CPU for processing, such as applying for a new memory space again	CORE _n _TRACE_INTR
TRACE_FIFO_OVERFLOW_INTR	Triggered when the internal FIFO overflows and one or more packets have been lost.	CORE _n _TRACE_INTR

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [2.9 Register Summary](#).

2.8 Programming Procedures

2.8.1 Encoder Option Configuration

The Encoder has some optional configuration options.

- Full address mode
 - The encoder uses delta address mode by default. Users can set [TRACE_FULL_ADDRESS](#) to enable full address mode for debugging.
- Restart after packet loss
 - Once the trace packet is lost due to FIFO overflow, the encoder will automatically stop if [TRACE_RESTART_ENA](#) is set. When the FIFO is empty, the encoder will restart to report packets.
- Stall CPU while FIFO is almost full
 - If [TRACE_STALL_ENA](#) is set, when the FIFO is almost full, a stall request will be asserted to the CPU, and the CPU will be stalled until the request is deasserted.
- Stop tracing when the CPU is halted

If [TRACE_HALT_ENA](#) is set, when the CPU is halted, the encoder will output a packet to report the address of the last instruction before halting, followed by a support packet to indicate that tracing has stopped. Upon the deassertion of the halted signal, the encoder will start tracing again, commencing with a synchronization packet.
- Stop tracing when the CPU is in reset

The encoder will be reset separately, if [TRACE_RESET_ENA](#) is set. When the CPU is in reset, the encoder will output a packet to report the address of the last instruction before reset, followed by a support packet to indicate that tracing has stopped. Once the CPU exits from reset, the encoder will start tracing again, commencing with a synchronization packet.

- Using trigger outputs from the Debug Module

The debug module of the RISC-V hart has a trigger unit. This defines a match control register ([mcontrol](#)) containing a 4-bit [action](#) field, and reserves codes 2 ~ 4 of this field for trace use. ESP32-P4 has implemented this feature. The values of the [action](#) field are listed in Table 2.8-1.

Table 2.8-1. Debug module trigger support (the [action](#) field of the [mcontrol](#) register)

Value	Description
2	Trace-on: start trace
3	Trace-off: end trace
4	Trace-notify: notify the encoder to report an address

For details about the debug module trigger unit, please refer to Chapter 1 [High-Performance CPU](#).

2.8.2 Filter Configuration

The filter unit described in Section 2.5.4 can be configured as follows:

1. Select one of the following match modes:

- Set [TRACE_MATCH_PRIVILEGE](#) to enable privilege match mode:
 - Configure [TRACE_MATCH_CHOICE_PRIVILEGE](#) to specify the privilege level.
- Set [TRACE_MATCH_ECAUSE](#) to enable ecause match mode:
 - Configure [TRACE_MATCH_CHOICE_ECAUSE](#) to specify the ecause value
- Set [TRACE_MATCH_INTERRUPT](#) to enable interrupt match mode:
 - Configure [TRACE_MATCH_VALUE_INTERRUPT](#) to specify interrupt level
- Set [TRACE_MATCH_COMP](#) to enable comparator match mode:
 - Configure the primary comparator:
 - * Configure [TRACE_P_INPUT](#) to choose the input
 - * Configure [TRACE_P_FUNCTION](#) to select the comparator function
 - * Configure [TRACE_FILTER_P_COMPARATOR_MATCH_REG](#) to define match value
 - Configure the secondary comparator if needed:
 - * Configure [TRACE_S_INPUT](#) to choose the input
 - * Configure [TRACE_S_FUNCTION](#) to select the comparator function
 - * Configure [TRACE_FILTER_S_COMPARATOR_MATCH_REG](#) to define match value
 - Choose which comparator to match via [TRACE_MATCH_MODE](#):
 - * 0: only the primary comparator matches, and the secondary comparator has no effect

- * 1: both the primary comparator and the secondary comparator match ($P \& \& S$)
- * 2: neither the primary comparator nor secondary comparator match $!(P \& \& S)$
- * 3: start filtering when primary matches up until secondary matches

If [TRACE_P_NOTIFY](#) or [TRACE_S_NOTIFY](#) is set, the comparator will report the matched address as a notification, the instructions will not be filtered. For example:

- If the [TRACE_MATCH_MODE](#) is 0, and [TRACE_P_NOTIFY](#) is set, then all instructions will be traced, and the matched primary comparator will be notified.
- If the [TRACE_MATCH_MODE](#) is 0, and [TRACE_S_NOTIFY](#) is set, the instruction will be filtered by the primary comparator, and the instruction that matches the secondary comparator will be notified.

2. Set [TRACE_FILTER_EN](#) to enable filter unit.

2.8.3 Enable Encoder

- Configure the address space for the trace memory via [TRACE_MEM_START_ADDR_REG](#) and [TRACE_MEM_END_ADDR_REG](#). The encoder can access L2 MEM and PSRAM:
 - L2MEM: 0x4FF00000 ~ 0x4FFBFFFF
 - PSRAM: 0x48000000 ~ 0x4BFFFFFF
- Update the value of [TRACE_MEM_CURRENT_ADDR_REG](#) to the value of [TRACE_MEM_START_ADDR_REG](#) by setting [TRACE_MEM_CURRENT_ADDR_UPDATE](#)
- (Optional) Configure the memory writing mode via the [TRACE_MEM_LOOP](#) bit of [TRACE_TRIGGER_REG](#)
 - 0: Non-loop mode
 - 1: Loop mode (default)
- Configure the synchronization mode via the [TRACE_RESYNC_MODE](#) bit of [TRACE_RESYNC_PROLONGED_REG](#)
 - 0: Disable the synchronization counter
 - 1: Invalid. No effect
 - 2: Synchronization counter counts by packet
 - 3: Synchronization counter counts by cycle
- (Optional) Configures the threshold for the synchronization counter (default value is 128) via [TRACE_RESYNC_PROLONGED_REG](#)
- (Optional) Enable Interrupt
 - Set the corresponding bit of [TRACE_INTR_ENA_REG](#) to enable the corresponding interrupt
 - Set the corresponding bit of [TRACE_INTR_CLR_REG](#) to clear the corresponding interrupt
 - Read [TRACE_INTR_RAW_REG](#) to know which interrupt occurs
- (Optional) Enable automatic restart by setting the [TRACE_RESTART_ENA](#) bit of [TRACE_TRIGGER_REG](#). This function is enabled by default

- There are 2 ways to enable the trace encoder:
 - Setting the [TRACE_TRIGGER_ON](#) bit of [TRACE_TRIGGER_REG](#)
 - Through the Debug module Trace-on to start the encoder. When the trigger is matched and the [action](#) is 2, it will start the encoder. For trigger configurations, please refer to the RISC-V Debug Specification.

Once the encoder is enabled, it will keep tracing the HP CPU's instruction trace interface and writing packets to the trace memory.

2.8.4 Disable Encoder

There are two ways to end output trace packets:

1. Set [TRACE_TRIGGER_OFF](#) of [TRACE_TRIGGER_REG](#) to end the encoder
2. Through the Debug module Trace-off to end the encoder. When the trigger is matched and the [action](#) is 3, it will end the encoder. For trigger configurations, please refer to the RISC-V Debug Specification.

Due to the internal FIFO, the packets are not written to the memory immediately after the end of the trace. It is recommended to query the [TRACE_FIFO_STATUS_REG](#) to confirm that the data is all written to memory.

Table 2.8-2. Trace Status

Field	Description
TRACE_FIFO_EMPTY	If 1 indicates that the FIFO is empty, all data have been written to memory
TRACE_WORK_STATUS	Encoder's work status: • 0: idle state, the encoder is not started • 1: the encoder is normally working that output packets • 2: the encoder is not outputting packets, the CPU is halted or in reset • 3: the encoder is not outputting packets, it occurs while a packet is lost, and is waiting for FIFO empty to restart.

2.8.5 Notify

Users may want to report a special address, even if it is not the target of an uninferable discontinuity. There are two ways to notify and report a special address:

1. The debug trigger unit matched, and the [action](#) is 4
2. The filter comparator matched

When a notification is requested, the encoder will report a format 2 or format 1 with an address packet, determined by the [branch_map](#). If [branch_map](#) is 0, report format 2, otherwise report format 1. The notify field shows whether the packet is reporting a notification address. If it is the target of an uninferable discontinuity, even if it is a notification, the notify field will not be asserted, otherwise the decoder cannot reconstruct the instruction stream correctly.

2.8.6 Decode Data Packets

- Find the first address to decode
 - Read the [TRACE_MEM_FULL_INTR_RAW](#) bit of the [TRACE_INTR_RAW_REG](#) register to know if the trace memory is full
 - * if read 0, read the trace packets from [TRACE_MEM_START_ADDR_REG](#)
 - * if read 1, and the loop mode is enabled, then the old trace packets are overwritten. In this case, read the [TRACE_MEM_CURRENT_ADDR_REG](#) to know the last writing address, and use this address as the first address to decode
- Use the decoder to decode data packets
 - The decoder reads all data packets starting from the first address, and reconstructs the data stream with the binary file
 - As mentioned in [2.6](#), the encoder writes 14 zero bytes to the memory partition boundary every time when 128 packets are transmitted. Given this fact, the first non-zero byte after 14 zero bytes should be the header of a new packet

2.8.7 AHB Configuration

ESP32-P4 encoder supports the AHB bus. There are some optional configurations to control the transmission bandwidth of the encoder.

If there are many DMA masters, users can increase the bandwidth of Trace by setting AHB burst [TRACE_AHB_CONFIG_REG](#).

- The [TRACE_HBURST](#) supports the following configurations:
 - 0x0: SINGLE
 - 0x1: INCR (length not defined)
 - 0x2: INCR4
 - 0x4: INCR8
 - Others: reserved

Burst transfer means that once arbitration is successful, multiple words can be transferred continuously. For SINGLE, the length is 1; for INCR, the length is undefined; for INCR4, the length is 4; for INCR8, the length is 8. The longer the burst length, the better the bandwidth.

- When configured as INCR transfer, since INCR is a burst of indeterminate length, in order to avoid Trace occupying the bus for a long time, it will end the INCR transfer after the transfer length up to [TRACE_MAX_INCR](#), and restart bus arbitration.

This is the configuration used to adjust the bandwidth. When the bandwidth is sufficient, it's not required to do the configuration, and users can use the default configuration.

2.8.8 Software Retention

The `TRACE_WORK_STATUS` represents the state of the encoder. When the encoder is not in the IDLE state, it must backup and restore the configuration when the chip enters sleep.

2.9 Register Summary

The addresses in this section are relative to RISC-V Trace Encoder base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

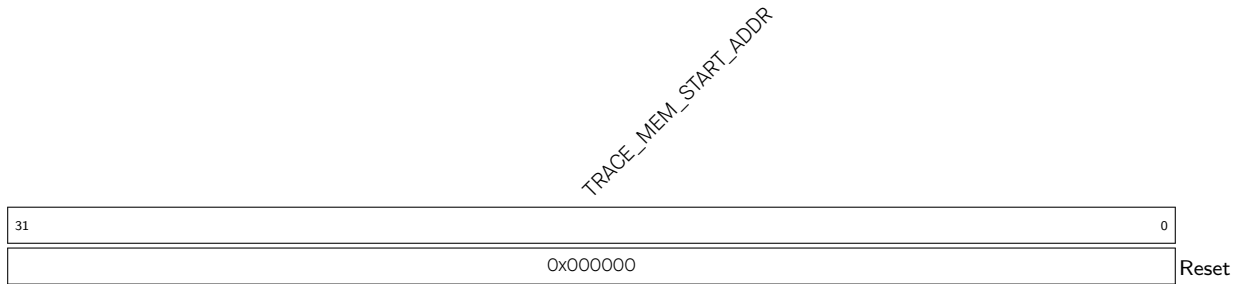
Name	Description	Address	Access
Memory configuration registers			
TRACE_MEM_START_ADDR_REG	Memory start address	0x0000	R/W
TRACE_MEM_END_ADDR_REG	Memory end address	0x0004	R/W
TRACE_MEM_CURRENT_ADDR_REG	Memory current address	0x0008	RO
TRACE_MEM_ADDR_UPDATE_REG	Memory address update	0x000C	WT
FIFO status register			
TRACE_FIFO_STATUS_REG	FIFO status register	0x0010	RO
Interrupt registers			
TRACE_INTR_ENA_REG	Interrupt enable register	0x0014	R/W
TRACE_INTR_RAW_REG	Interrupt raw status register	0x0018	RO
TRACE_INTR_CLR_REG	Interrupt clear register	0x001C	WT
Trace configuration registers			
TRACE_TRIGGER_REG	Trace trigger register	0x0020	varies
TRACE_CONFIG_REG	Trace configuration register	0x0024	R/W
TRACE_FILTER_CONTROL_REG	Filter control register	0x0028	R/W
TRACE_FILTER_MATCH_CONTROL_REG	Filter match control register	0x002C	R/W
TRACE_FILTER_COMPARATOR_CONTROL_REG	Filter comparator match control register	0x0030	R/W
TRACE_FILTER_P_COMPARATOR_MATCH_REG	Primary comparator match value register	0x0034	R/W
TRACE_FILTER_S_COMPARATOR_MATCH_REG	Secondary comparator match value register	0x0038	R/W
TRACE_RESYNC_PROLONGED_REG	Resynchronization configuration register	0x003C	R/W
TRACE_AHB_CONFIG_REG	AHB configuration register	0x0040	R/W
Clock gating control and configuration register			
TRACE_CLOCK_GATE_REG	Clock gating control register	0x0044	R/W
Version register			
TRACE_DATE_REG	Version control register	0x03FC	varies

2.10 Registers

The addresses in this section are relative to RISC-V Trace Encoder base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

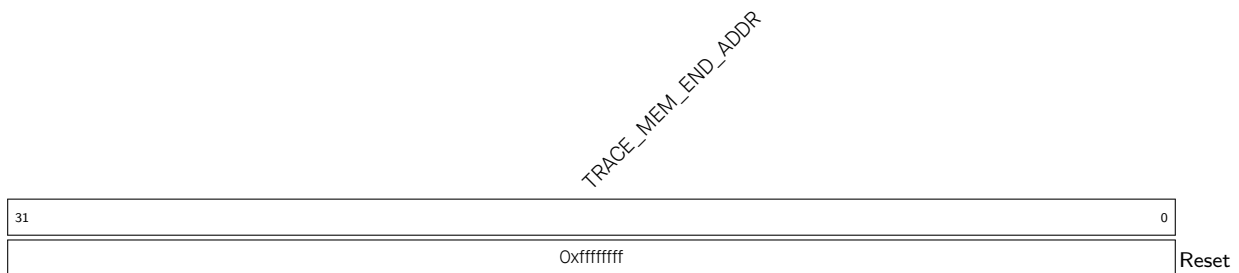
For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 2.1. TRACE_MEM_START_ADDR_REG (0x0000)



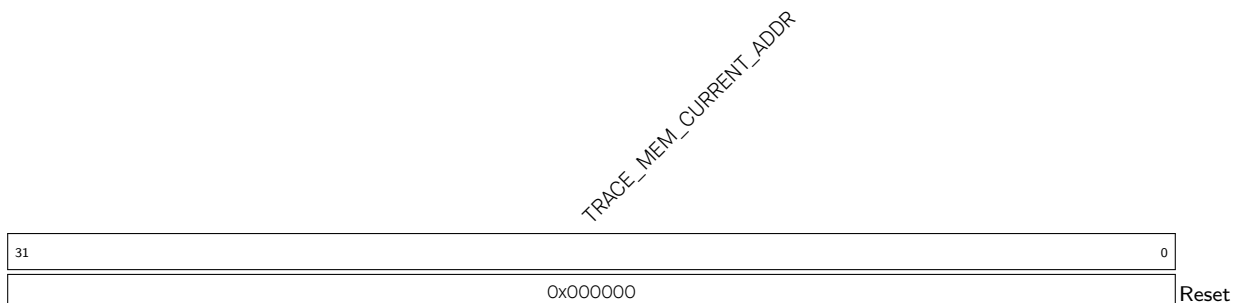
TRACE_MEM_START_ADDR Configures the start address of the trace memory. (R/W)

Register 2.2. TRACE_MEM_END_ADDR_REG (0x0004)



TRACE_MEM_END_ADDR Configures the end address of the trace memory. (R/W)

Register 2.3. TRACE_MEM_CURRENT_ADDR_REG (0x0008)



TRACE_MEM_CURRENT_ADDR Represents the current memory address for writing. (RO)

Register 2.4. TRACE_MEM_ADDR_UPDATE_REG (0x000C)

Diagram of the 32-bit TRACE_MEN register structure:

- Bit 31 to Bit 1: (reserved)
- Bit 0: TRACE_MEN
- Reset: 0

TRACE_MEM_CURRENT_ADDR_UPDATE Configures whether to update the value of `TRACE_MEM_CURRENT_ADDR` to `TRACE_MEM_START_ADDR`.

0: Not update

1: Update

(WT)

Register 2.5. TRACE_FIFO_STATUS_REG (0x0010)

[illegible]

TRACE_FIFO_EMPTY Represents whether the FIFO is empty.

1: Empty

0: Not empty (RO)

TRACE_WORK_STATUS Represents the state of the encoder:

0: Idle state

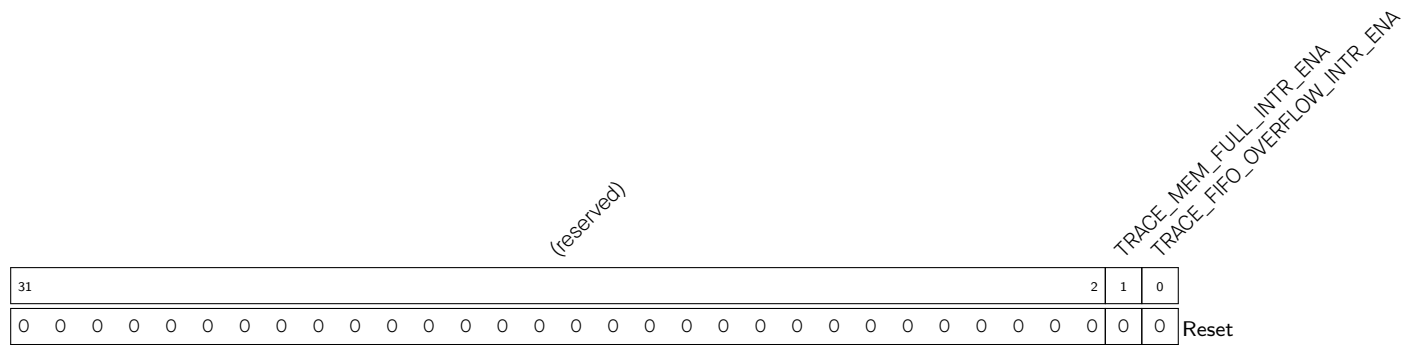
1: Working state

2: Wait state because the hart is halted or in reset

3: Lost state

(RO)

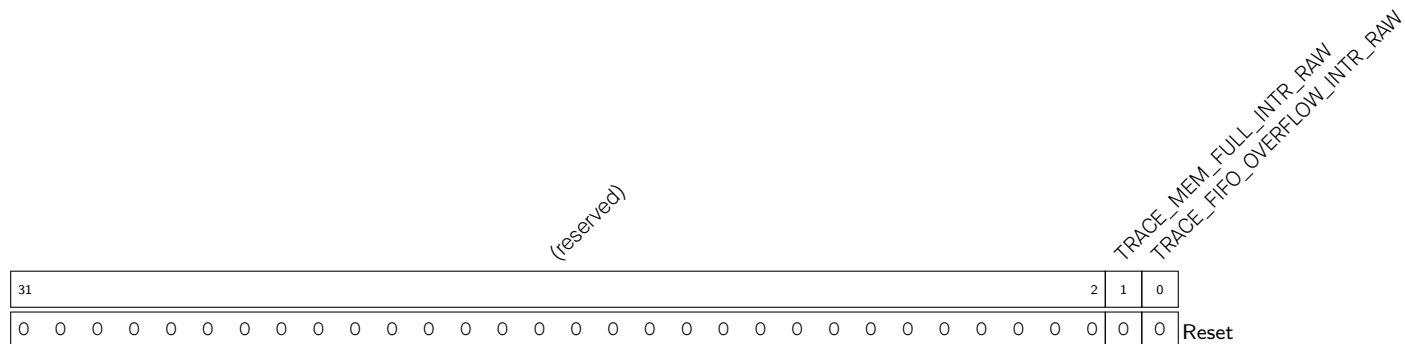
Register 2.6. TRACE_INTR_ENA_REG (0x0014)



TRACE_FIFO_OVERFLOW_INTR_ENA Write 1 to enable TRACE_FIFO_OVERFLOW_INTR. (R/W)

TRACE_MEM_FULL_INTR_ENA Write 1 to enable TRACE_MEM_FULL_INTR. (R/W)

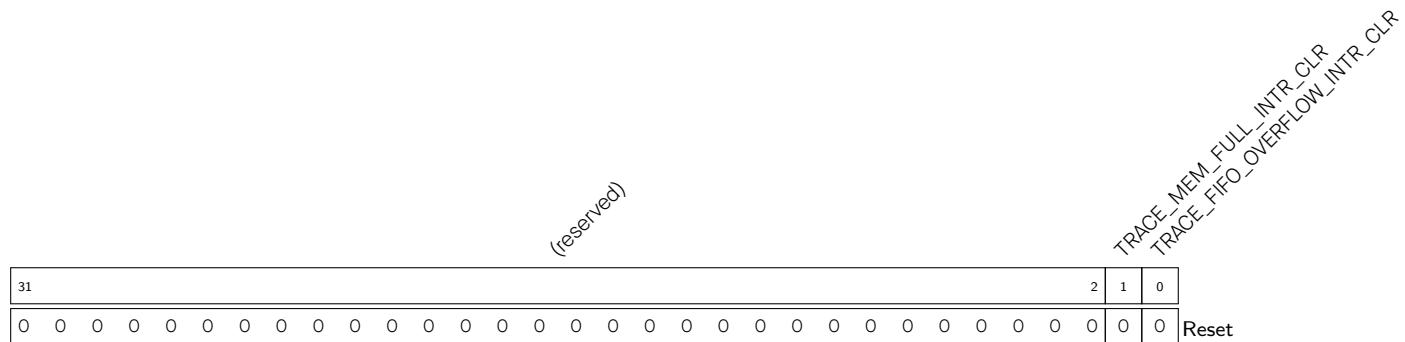
Register 2.7. TRACE_INTR_RAW_REG (0x0018)



TRACE_FIFO_OVERFLOW_INTR_RAW The raw interrupt status of TRACE_FIFO_OVERFLOW_INTR.
(RO)

TRACE_MEM_FULL_INTR_RAW	The raw interrupt status of TRACE_MEM_FULL_INTR. (RO)
--------------------------------	---

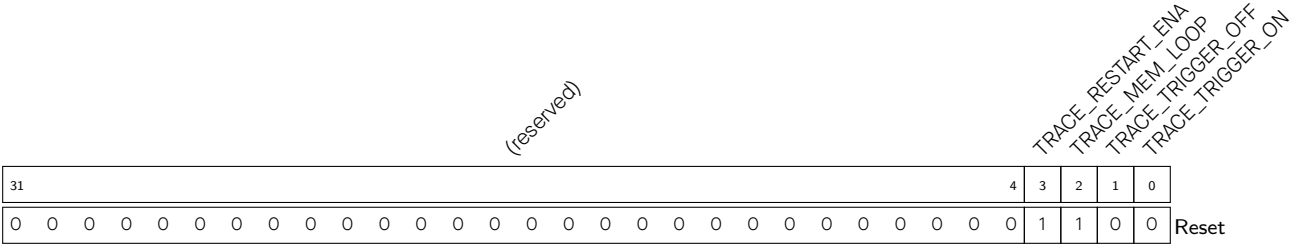
Register 2.8. TRACE_INTR_CLR_REG (0x001C)



TRACE_FIFO_OVERFLOW_INTR_CLR Write 1 to clear TRACE_FIFO_OVERFLOW_INTR. (WT)

TRACE_MEM_FULL_INTR_CLR Write 1 to clear TRACE_MEM_FULL_INTR. (WT)

Register 2.9. TRACE_TRIGGER_REG (0x0020)



- TRACE_TRIGGER_ON

Configures whether to enable the encoder.

0: Invalid

1: Enable

(WT)
- TRACE_TRIGGER_OFF

Configures whether to disable the encoder.

0: Invalid

1: Disable

(WT)
- TRACE_MEM_LOOP

Configures the memory writing mode.

0: Non-loop mode

1: Loop mode

(R/W)
- TRACE_RESTART_ENA

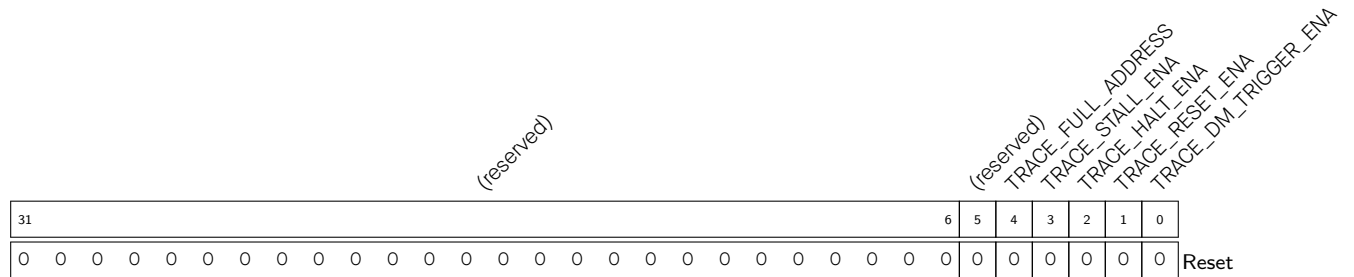
Configures whether to enable automatic restart.

0: Disable

1: Enable

(R/W)

Register 2.10. TRACE_CONFIG_REG (0x0024)



TRACE_DM_TRIGGER_ENA Configure whether to enable the trigger signal.

0: Disable

1: Enable

(R/W)

TRACE_RESET_ENA Configures whether to enable the reset signal.

0: Disable

1: Enable

(R/W)

TRACE_HALT_ENA Configures whether to enable the halted signal.

0: Disable

1: Enable

(R/W)

TRACE_STALL_ENA Configures whether to enable the stall signal.

0: Disable

1: Enable

(R/W)

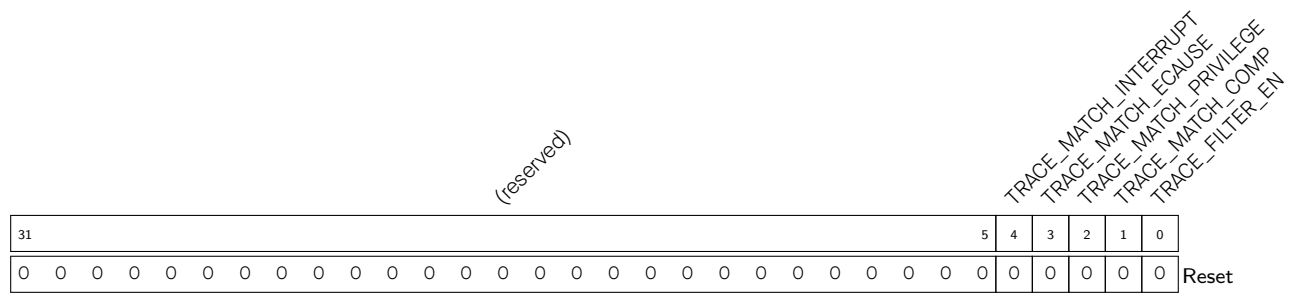
TRACE_FULL_ADDRESS Configure the address mode.

0: Delta address mode

1: Full address mode

(R/W)

Register 2.11. TRACE_FILTER_CONTROL_REG (0x0028)



TRACE_FILTER_EN Configure whether to enable filtering.

0: Disable

1: Enable

(R/W)

TRACE_MATCH_COMP Configures whether to enable the comparator match mode.

0: Disable

1: Enable

(R/W)

TRACE_MATCH_PRIVILEGE Configures whether to enable the privilege match mode.

0: Disable

1: Enable

(R/W)

TRACE_MATCH_ECAUSE Configures whether to enable the ecause match mode.

0: Disable

1: Enable

(R/W)

TRACE_MATCH_INTERRUPT Configures whether to enable the interrupt match mode.

0: Disable

1: Enable

(R/W)

Register 2.12. TRACE_FILTER_MATCH_CONTROL_REG (0x002C)

The diagram shows the bit fields of the Interrupt Cause Register (ICR) as follows:

- Reserved:** Bits 31 down to 8 are reserved.
- TRACE_MATCH_CHOICE_ECAUSE:** A 7-bit field located at bits 7 through 1.
- TRACE_MATCH_VALUE_INTERRUPT_PRIVILEGE:** A 4-bit field located at bits 0 through 3.

TRACE_MATCH_CHOICE_PRIVILEGE Configures the privilege level for matching.

0: User mode

1: Machine mode

(R/W)

TRACE_MATCH_VALUE_INTERRUPT Configures the interrupt level for matching. Valid only when

TRACE_MATCH_INTERRUPT is set.

0: itype=1

1: itype=2

(R/W)

TRACE_MATCH_CHOICE_ECAUSE Configures the ecause code for matching. (R/W)

Register 2.13. TRACE_FILTER_COMPARATOR_CONTROL_REG (0x0030)

(reserved)																TRACE_MATCH_MODE		(reserved)		TRACE_S_NOTIFY		TRACE_S_FUNCTION		(reserved)		TRACE_S_INPUT		(reserved)		TRACE_P_NOTIFY		TRACE_P_FUNCTION		(reserved)		TRACE_P_INPUT	
31																18	17	16	15	14	13	12	10		9	8	7	6	5	4	2		1	0			
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0 0 0		0		0		0	0	0	0	0	0		0	0	Reset					

TRACE_P_INPUT Configures the input of the primary comparator for matching.

0: iaddr

1: tval

(R/W)

TRACE_P_FUNCTION Configures the function for the primary comparator.

0: Equal

1: Not equal

2: Less than

3: Less than or equal

4: Greater than

5: Greater than or equal

Others: Always match

(R/W)

TRACE_P_NOTIFY Configures whether to explicitly report an instruction address matched against the primary comparator.

0: Not report

1: Report

(R/W)

TRACE_S_INPUT Configures the input of the secondary comparator for matching.

0: iaddr

1: tval

(R/W)

TRACE_S_FUNCTION Configures the function of for secondary comparator.

0: Equal

1: Not equal

2: Less than

3: Less than or equal

4: Greater than

5: Greater than or equal

6: The second comparator value is the masked value of the primary comparator

Others: Always match

(R/W)

Continued on the next page...

Register 2.13. TRACE_FILTER_COMPARATOR_CONTROL_REG (0x0030)

Continued from the previous page...

TRACE_S_NOTIFY Configures whether to explicitly report an instruction address matched against the secondary comparator.

0: Not report

1: Report

(R/W)

TRACE_MATCH_MODE Configures the comparator match condition.

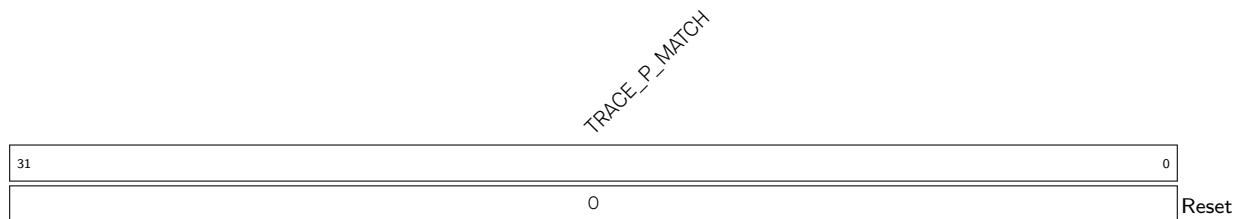
0: Only the primary comparator matches

1: Both primary and secondary comparators match (P&&S)

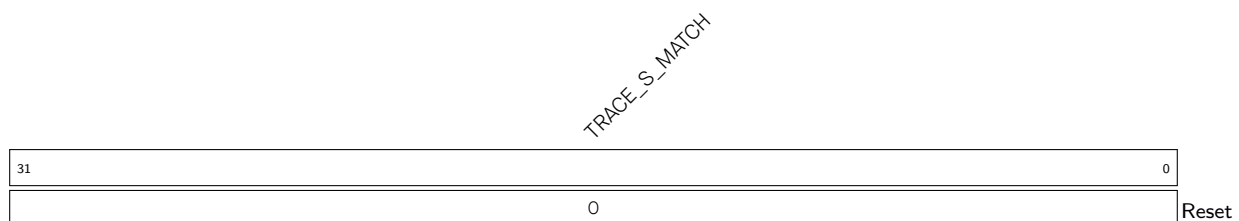
2: Neither primary nor secondary comparator match (!P&&S)

3: Start filtering when the primary comparator matches and stop filtering when the secondary comparator matches

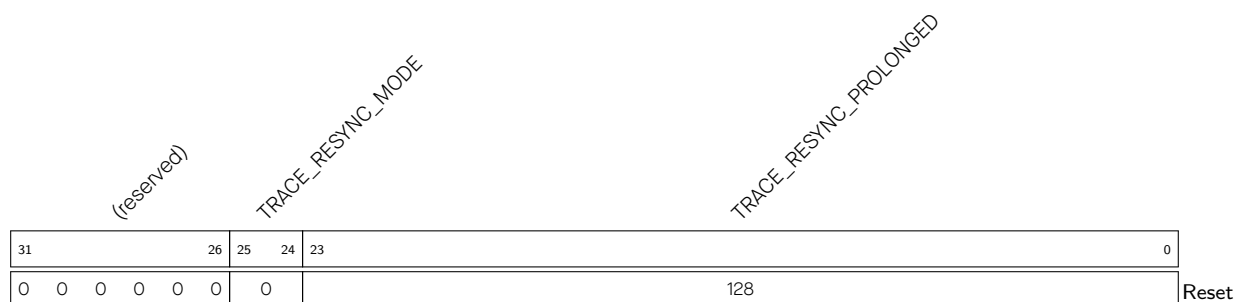
(R/W)

Register 2.14. TRACE_FILTER_P_COMPARATOR_MATCH_REG (0x0034)

TRACE_P_MATCH Configures the match value for the primary comparator. (R/W)

Register 2.15. TRACE_FILTER_S_COMPARATOR_MATCH_REG (0x0038)

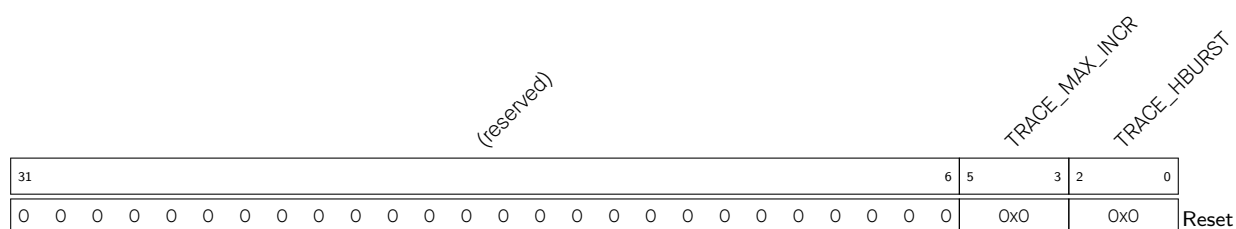
TRACE_S_MATCH Configures the match value for the secondary comparator. (R/W)

Register 2.16. TRACE_RESYNC_PROLONGED_REG (0x003C)

TRACE_RESYNC_PROLONGED Configures the threshold for the synchronization counter. (R/W)

TRACE_RESYNC_MODE Configures the synchronization mode.

- 0: Disable the synchronization counter
 - 1: Invalid
 - 2: Synchronization counter counts by packet
 - 3: Synchronization counter counts by cycle
- (R/W)

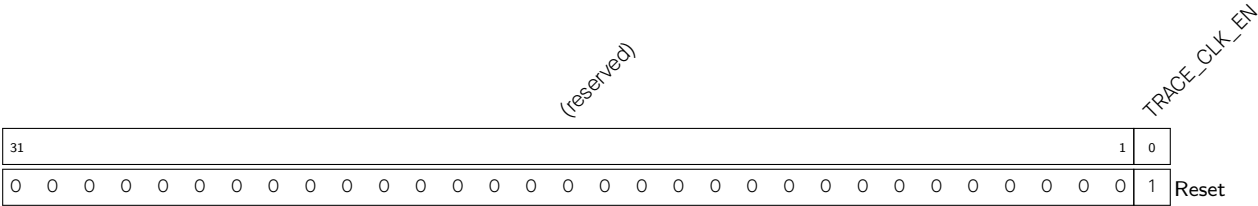
Register 2.17. TRACE_AHB_CONFIG_REG (0x0040)

TRACE_HBURST Configures the AHB burst mode.

- 0: SINGLE
 - 1: INCR (length not defined)
 - 2: INCR4
 - 4: INCR8
 - Others: Invalid
- (R/W)

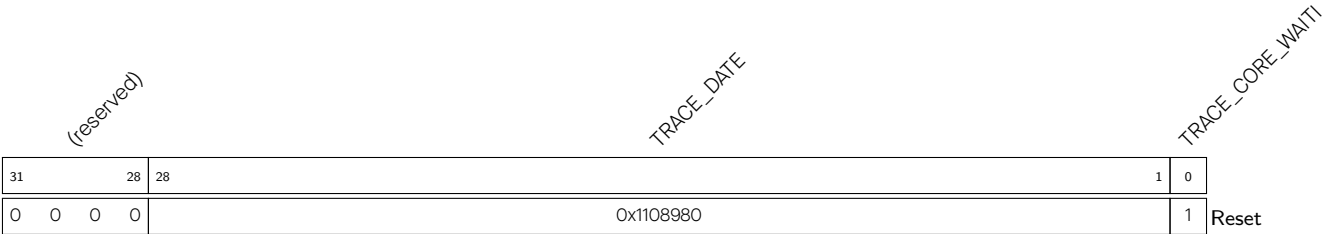
TRACE_MAX_INCR Configures the maximum burst length for INCR mode. (R/W)

Register 2.18. TRACE_CLOCK_GATE_REG (0x0044)



TRACE_CLK_EN Configures register clock gating. 0: Support clock only when the application writes registers to save power
1: Always force the clock on for registers
This bit doesn't affect register access
(R/W)

Register 2.19. TRACE_DATE_REG (0x03FC)



TRACE_DATE Version control register. (R/W)

TRACE_CORE_WAITI Represents whether the HP CPU cores are in the Wait-For-Interrupt (WFI) state.
0: Not in WFI
1: In WFI
(RO)